Week 3: Some Other Topics and Case Study of Temp and Humidity Sensor Project

121

Neil Cherry, BS-CS

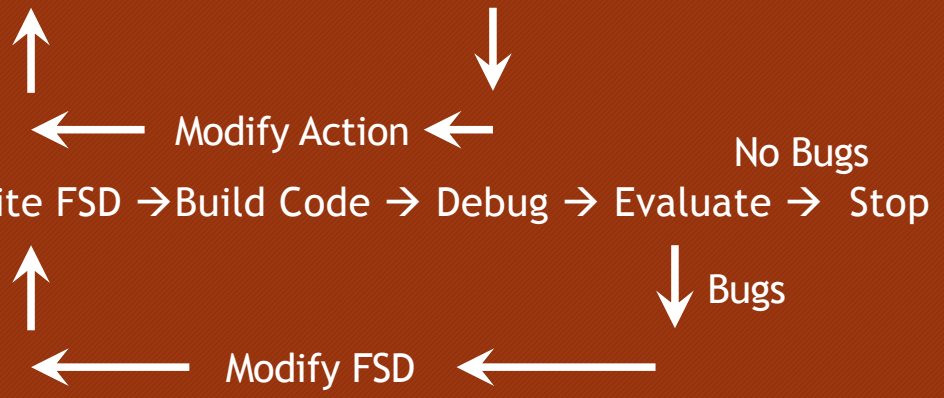
Joseph E. Wilkes, PhD-EE

September 2026

Questions from Last Week

Types of AI Flow

- (Simple) Chat
  - Ask → Answer → Stop
    - Ex: Current Weather→ Raining
- Agent
  - Ask → Action(one or more steps) → Stop
    - Ex: Compile Code Base
- Agentic
  - Ask → Action → Result → Evaluate → Stop



ESP32-S3 Development (CLI Workflow)

- The ESP32 microprocessor series has become popular lately
- Espressif Systems makes the chips and has its own development environment which can be installed in the Container

Case Study: Agent Based (Human in Loop)

- Goals:
  - Develop a Temperature, Humidity and Atmospheric Pressure Sensor
  - Develop the project without writing any code
- Why:
  - I am not as strong in C++ as I am with other languages
  - I am not familiar with the ESP32 development environment
  - Have not written a web page since the early 90s
- How:
  - See the following slides

Build an FSD_creator skill

- Used Claude skill_creator skill
- Did this on my Mac Mini using Claude.ai
- Later installed Claude app on Mac Mini and Windows laptop
- I also installed Claude Code in a Docker Container on my Windows laptop

Description

- name: fsd-creator
- description: >
- Generates a Feature Specification Document (FSD) in Markdown format from a high-level project description. Use this skill whenever the user says "createfsd", "newproject", or "newrequirements", or asks to generate a feature spec, requirements document, projectspec, or FSD. Also trigger when the user describes a new software project and wants documentation, requirements, or a spec produced — even if they don't use those exact keywords.
- Always use this skill when any of these triggers appear, even mid-conversation.

Conduct the Interview

Core questions to ask (as applicable):

1. **Project name** — What is this project called?
2. **Platform / target hardware** — What hardware or OS is this running on?(e.g., STM32F4, Raspberry Pi, Windows x64, bare-metal, RTOS, Linux)
3. **Language / toolchain** — Primary programming language(s) and compiler/IDE?
4. **Top-level goals** — What are the 2–4 most important things this system must do?
5. **External interfaces** — What does this system talk to? (sensors, APIs, buses like SPI/I2C/UART, cloud services, other software modules, etc.)
6. **Constraints** — Any hard constraints? (memory limits, timing/latency, power budget, safety requirements, coding standards like MISRA, etc.)
7. **Out of scope** — Anything explicitly NOT part of this project?
8. **Revision info** — Author name and initial version number (default: v0.1)?

Only ask what isn't already answered by the description. If the project description is thorough, you may only need to ask 2–3 follow-up questions.

FSD Document Structure

- [Project Name] — Feature Specification Document
- Revision History
- 1. Project Overview
- 2. Functional Requirements
- 3. Hardware & Platform Constraints
- 4. API & Interface Definitions
- 5. Test Plans
- Appendix (optional)

My Inputs (via a file I created as a .md)

- Project: ESP32 Sensor Monitor
- Target Hardware: ESP32-S3
- Language: C++
- Top Level Goals:
- Goal 1 When the ESP starts for the first time it should generate a WiFi access point that can be used to enter the SSID and password for a different WiFi network.
- Goal 2 After the user enters the data and clicks on a connect to remote WiFi button, the
- ESP-32 should reboot and connect to the remote WiFi network.
- Goal 3 It should then collect the temperature, humidity and atmospheric pressure every 5 seconds, date and time from ntp and display it on a web site connected to the network.
- Goal 4 It should also graph the data every half hour for the last 24 hours.
- External Interfaces: BME280
- Constraints: None
- Out of Scope: None
- Revision info Author Joseph Wilkes, version number 1.0

Appendix — Open Questions / TBD Items

1. Final I2C pin assignment for SDA/SCL on the chosen ESP32-S3 dev board.
2. Whether to include a physical "factory reset" button (long-press GPIO) to clear NVS credentials — currently relies on STA-failure fallback (REQ-007).
3. Status LED behavior (solid/blink patterns) for AP vs STA vs error states.
4. NTP server choice — default `pool.ntp.org`; consider region-specific or RTC-fallback.
5. Web asset storage: bake into firmware vs. LittleFS partition.

Response to Action Items

- Gave inputs to fix action items and iterated from V2 to V3 to V4. when it responded with updated V4, I asked Claude if I could create code from FSD
- It identified developer options that I needed to nail down. Nailed down some; added some; let a few go to default; got version V5
- Asked it if V5 was ready to send to developers; it said yes.
- Next step try to get Claude to write code.

Next steps with Claude Code running in Docker

- Build Session v1 — ESP32 Sensor Monitor**Date:** 2026-05-23 **Platform:** WSL2 Linux (ESP-IDF v5.4.1, ESP32-S3)
- Exchange 1:
- User:
  - install The Espressif IoT Development Framework (ESP-IDF) for the esp32-s3
- Claude:
  - Checked prerequisites and environment.
  - Found ESP-IDF v5.4.1 already installed at `/opt/esp/idf` with the ESP32-S3 Xtensa toolchain (`xtensa-esp-elf-gcc 14.2.0`)

I had previously installed the IDE; but forgot I did

Next Exchange

- **User:
  - build the project described in esp32-sensor-monitor-fsd-v5.md; if you are running out of resources save you work so it can be restarted in a later session when resources are available**
- Claude:
  - Read `esp32-sensor-monitor-fsd-v5.md` (FSD v5.0 — 539 lines). Fixed `/CDL_AI_class` permissions (`sudo chmod 777`), then created the full project at `/CDL_AI_class/esp32_sensor_monitor/`.

Results

- Bunch of output will Claude describes what it was doing
- Finally: Build result
- Project build complete
  - esp32_sensor_monitor.bin binary size 0xe2e50 bytes (908 KB).
  - Smallest app partition is 0x200000 bytes. 0x11d1b0 bytes (56%) free.
  - Bootloader binary size 0x5220 bytes. 0x2de0 bytes (36%) free.```

Next Steps

- **User**: flash it
- **Claude**:
- Detected user `node` not in `dialout` group.
- Ran:```bash sudo usermod -aG dialout node sudo chmod a+rw /dev/ttyACM0```
- Flashed firmware:```idf.py -p /dev/ttyACM0 flash``````
- Chip is ESP32-S3 (QFN56) (revision v0.2)
- Features: WiFi, BLE, Embedded PSRAM 8MB (AP_3v3)MAC: e0:72:a1:d4:ff:34
- Wrote 929360 bytes (580302 compressed) at 0x00020000 in 12.1 seconds Hash of data verified.
- Hard resetting via RTS pin...
- Done```
- Flashed SPIFFS:```python -m esptool --chip esp32s3 -p /dev/ttyACM0 -b 460800 write_flash 0x220000 build/spiffs.bin``````
- Wrote 1048576 bytes (83769 compressed) at 0x00220000 in 3.8 seconds Hash of data verified.
- Hard resetting via RTS pin...```
- Reported: Device rebooted.
- On first boot it will broadcast `ESP32-Monitor-FF34`.
- Connect to it — captive portal will present the provisioning form.
- After entering WiFi credentials the device reboots and the dashboard is available at `http://esp32.local/`.

On to Debugging: Debugging Round 1

- User:
  - the esp32 is running and displaying a web site with the requested data; however, humidity data is always 0.0 %; fix this; change temperature display to degrees F**
- Claude:
  - …
  - Built firmware with `idf.py build` — succeeded, 929 KB binary. Regenerated SPIFFS with `spiffsgen.py 0x100000 spiffs_data build/spiffs.bin` — 1.0 MB image.
- User: flash it
- Claude:
  - …
  - Hard resetting via RTS pin...

Debugging : Round 2

- User:
  - the device is not syncing to ntp; fix this**
- Claude:
  - Serial monitor diagnosis
  - …
  - Flash and verification
  - …
  - Serial monitor confirmed sync:```[+0.0s] I (2109) wifi_station: SNTP started — servers: time.nist.gov, pool.ntp.org, time.google.com[+2.8s] I (5159) wifi_station: NTP synced: 2026-05-26T15:04:42Z```NTP synced in **2.8 seconds** (via `pool.ntp.org` fallback).

Debugging: Round 2

- User:
  - make change to display time in local time not zulu
- Claude:
  - Asked user for timezone. User selected US Eastern (ET).
  - Built and flashed firmware.
  - Serial monitor confirmed:```I (22419) wifi_station:
  - NTP synced: 2026-05-26 11:08:37 EDT

Debugging: Round 3

- User:
  - claude the esp32 crashed around 2 hours of running can you find the bug
- Claude:
  - Analyzed all source files. Found three bugs:
  - Fixed bugs
- User:
  - Build it and flash it
- Claude:
  - Flash result: **success**- Chip: ESP32-S3 (QFN56, rev 0.2, 8MB PSRAM, 40MHz crystal)- MAC: `e0:72:a1:d4:ff:34`- All four partitions written and hash-verified: bootloader, app, partition table, OTA data-
  - Hard reset issued —
  - device booted new firmware

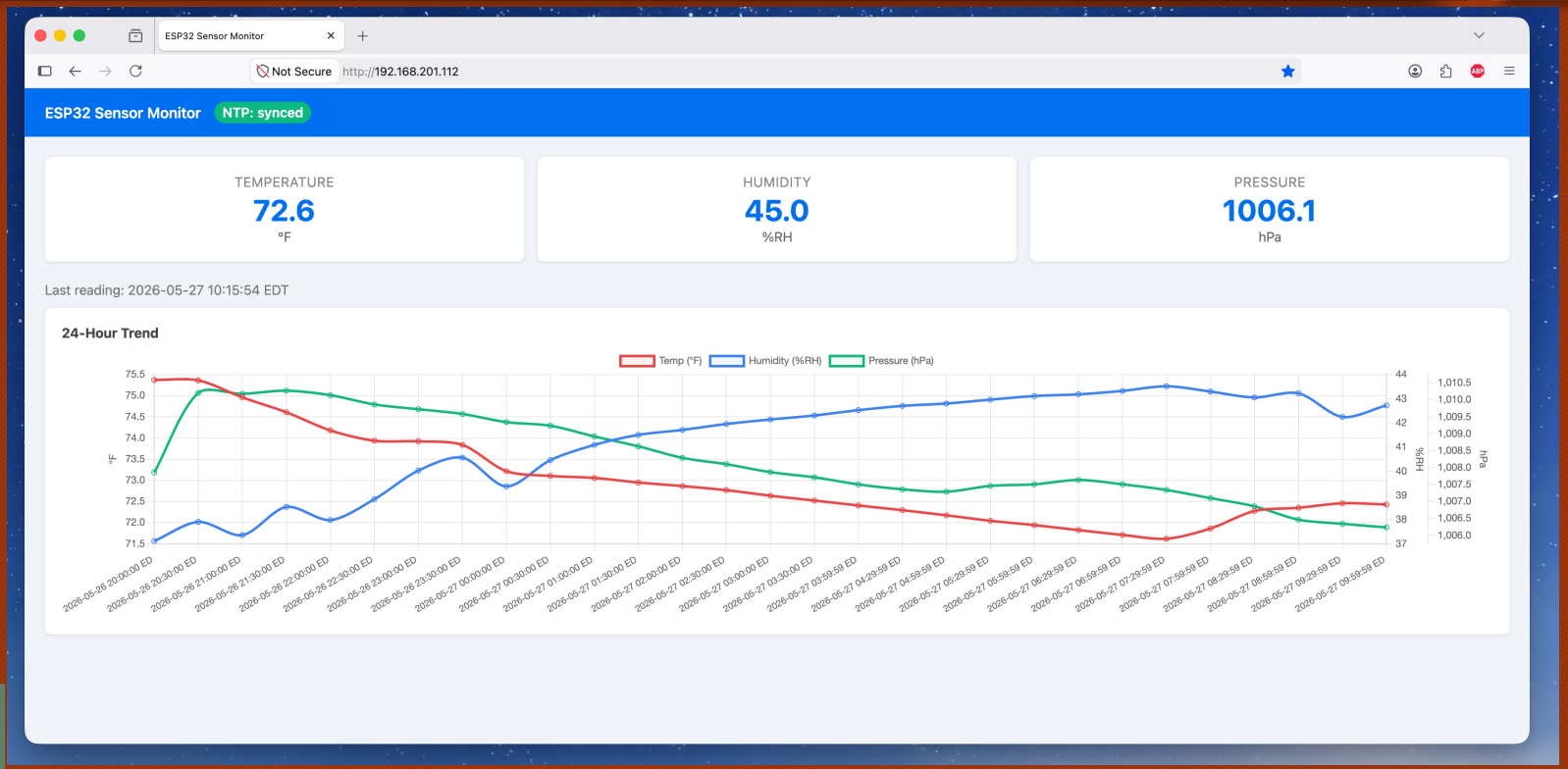
Debugging: Round 4 (Session data lost)

- User:
  - 24 hour chart does not update after 24 hours
- Claude
  - Spent an entire afternoon examining the code
  - Timed out on max tokens
  - Started again
  - Did this multiple times
  - No Results!!!
  - Could not find the bug

Debugging: Round 4 again

- User
  - Claude when the 24 graph is full fix the code so that the oldest data is deleted, the data shifted and the newest data is added at the end
- Claude
  - …
  - After rebuilding and flashing, once the buffer fills past 48 slots the chart will slide: oldest point drops off the left, newest appends on the right
- Project Now successfully works

Web page on my local network



Debugging Complex Code

- I have been having on going discussions on AI with my son who is a game programmer (BS-Math, BS-CS)
- I asked him how complex software is debugged
- He said by the Use of Telemetry (AKA debug logs) from the Code
- Claude included in its software for the Sensor a large amount of data streamed over the Serial Port
- I never looked at it but at times Claude did look at it to fix bugs

Summary

- User
  - how many lines of code are in this project
- Claude
  - Your code: 1,841 lines across 28 files.
  - The project also pulls in ~18,500 lines of third-party managed components (mostly the mDNS library from Espressif), but none of that is code you wrote.
- Goal Met: Built project without writing any code
- However: If I did not have previous development experience in multiple languages, I am not sure I could have told Claude in a way it understood how to find the last bug

Reverse Engineering a Code Base

- Often, we are given someone else's code to review or modify
- Claude can be used to do this very rapidly
- The example here is from a Raspberry Pi Pico Project from 2 years ago
  - Article: https://hackaday.com/2024/08/12/pi-pico-sdr-on-a-breadboard/
  - Project: https://101-things.readthedocs.io/en/latest/breadboard_radio.html
  - Github site for Code: https://github.com/dawsonjon/PicoRX
  - Complete Claude Review: https://github.com/Computer-Deconstruction-Lab/Software_Development_using_AI/blob/main/PicoRX_Code_Guide_with_questions.pdf

CDL

INFOAGE SCIENCE AND HISTORY MUSEUMS

The Project: Raspberry Pi Pico SDR



The Problem

- The Receiver worked but there was a buzz in the audio output
- I had one idea on what caused the problem but tabled the project until the discussion of Software Radios came up last week
- So, I asked Claude: “Analyze this code and provide a summary of the code so that I can make modifications to it. Do not report on any possible errors. Provide the output in a pdf format”
- The result was a 14-page review
- I asked a follow up question which is included in the review

CDL

INFOAGE SCIENCE AND HISTORY MUSEUMS

Review of Claude's Analysis

- Read Complete Claude Review: https://github.com/Computer-Deconstruction-Lab/Software_Development_using_AI/blob/main/PicoRX_Code_Guide_with_questions.pdf
- We will now switch to this pdf

Major Limitation of AI Systems

- The book I read in the 80s on Expert Systems focused on problems in building them
- It did not discuss how to code the Expert System
- A major Limitation at the time and is still true Today is:

AI Systems Lack Intuition!!

CDL

INFOAGE SCIENCE AND HISTORY MUSEUMS

What is Intuition?

- The power or faculty of attaining to direct knowledge or cognition without evident rational thought and inference
- Driving: Light Turns green; car on road with red fails to stop; Human senses this well before car gets near intersection and does not move into intersection and avoids a fatal Crash. How?
- Job Interviews: You have a strong feeling that an interview candidate isn't the right fit for your company culture, though they meet all the necessary qualifications on paper. How?
- Sports: Players know exactly where to move on field to intercept a play. How?

CDL

INFOAGE SCIENCE AND HISTORY MUSEUMS

Questions

Lab work

- Continue working on your class project
- Instructors will help with any problems